

Tool Migration Pre-conditions

This document outlines preconditions that are to be satisfied for the tool to be migrated from **SANDBOX** to **engineering-codebase**.

Responsibility

All checklist items mentioned in this document are the responsibility of the tool author and must be completed before migration request.

Info

The goal of these preconditions are to ensure **seamless** and **time efficient** migration.

Folder Structure

Consistent structuring of folders helps for easier maintenance in the long run.

- Recreate the folder structure if it does not follow the format given below.
- Ensure proper separation of:
 - Source code
 - Tests
 - Documentation
 - Configuration files

An example folder structure recommended by **DT** is as follows:

```
TOOLNAME
├─ TOOLNAME.m          # Source code (Main class)
├─ Templates           # Input templates
├─ changelog.yml       # Changelog yaml file
├─ docs                # Tool Documentation
├─ mkdocs.yml          # Configuration file for online documentation
├─ +toolname           # Tool package
│   └─ +utils           # Utility/supporting functions
│       └─ +tests       # Test setup (Unit/Integration tests)
```

Dependencies

Info

Dependencies must be documented to ensure that the tool does not rely on any modules outside the scope of the engineering codebase.

Understanding and tracking dependencies is essential for ensuring compatibility, maintainability, and smooth integration with the engineering codebase.

MATLAB Dependencies

The built-in MATLAB function `requiredFilesAndProducts` can be used to identify required files and products for any script or function.

This is useful for:

- Identifying dependent files across repositories.
- Listing required MATLAB toolboxes.
- Validating independence from non-engineering codebase modules.

For more information and example usage, please refer to [code-dependencies](#).

Action Items

If any dependencies are found within the sandbox repository or outside the engineering codebase scope, document them and refactor the usage of the associated scripts accordingly.

External Services

Document the items listed below before migration to ensure there are no potential blockers (such as : proxy, Zscaler, or missing libraries).

- **External Applications / Libraries**
 - Any new applications, MATLAB toolboxes, or libraries that must be installed separately.
 - External databases, external libraries, shared network drives etc.
- **Network and Internet Access**
 - Required internet connectivity or specific domains/URLs that need whitelisting.
 - If exceptions are needed (e.g., *opening of port, whitelisting domain in Zscaler, etc.*), raise an IT ticket and document the approval (approach DT for support).
 - Ensure tools can function with **Zscaler enabled**.
- **Connectivity Validation**
 - Test execution of the tool **with and without proxy**, on both **laptop** and **VDI**.
 - Test from **office** and **home networks** to confirm consistent behavior.

Tip

When listing external dependencies, include:

- Exact tool/library names and versions
- Installation instructions of external files
- Notes on connection validation

Reference Test Cases

Create test cases to verify the tool's functionality and store all the results (preferably in shared drives `R:\Initialer`) which will serve as a comparison basis after migration.

- **Number of Test Cases**
 - At least **two reference test cases ($N \geq 2$)** must be created.
 - Add more if the tool has very different execution paths.
- **Test Case Documentation**
 - Provide a short description for each case.
 - Specify **input files** and where they are located.
 - Define expected outputs (e.g., result files, logs, figures).
- **Execution Instructions**

- A short writeup on how to run the tool for each test case.
- **Environment Coverage**
 - Confirm that test cases run successfully on:
 - **Local laptop**
 - **2D Persistent VDI**
 - **2D Engineering VDI**

Code Coverage Report

Generate code coverage report for the test cases and `Unittests` if already available.

Option 1 (`Unittest` plugin)

- Code coverage report for `Unittest` alone (if already present) can be created by using MATLAB's built-in code coverage plugin in `Unittest` setup. This will generate an interactive `.html` file.

Example:

```
% run set of tests including coverage report
Unittest.run('toolname', 'unit', '', 'RunCoverage', true)
```

Option 2: (`TestRunner` + `CodeCoveragePlugin`)

- Coverage report for test cases created [here](#) along with unit test, can also be generated using `TestRunner` and `CodeCoveragePlugin`
- For detailed information on how to generate the coverage report please refer to [coverage testing](#).

Note

The *README* for coverage report creation in Option 2 is tailored to a specific tool. Please adjust it as needed to fit the requirements of the current tool..

Run Pre-commit Hooks (SANDBOX)

Run pre-commit hooks on all files before migration and fix as many errors as possible.

Note

It is recommended to run pre-commit hooks and clear the errors while still in **SANDBOX**, as this significantly reduces error propagation during migration.

To run pre-commit hooks follow the steps below depending on your setup

With engineering-codebase dev setup

1. Make the necessary code changes.
2. Open powershell

```
cd <location to the tool base folder>
```

3. Run hooks on all files:

```
pre-commit run --files $(git ls-files <path-to-tool-folder-with-files>/*)
```

4. Fix reported issues and repeat until the hooks pass.

Without engineering-codebase dev setup

1. Verify if python is installed: run the following command in `Terminal / bash / powershell`

```
python --version
```

If python is already installed, the following message is shown:

```
Python 3.11.5
```

If the above information is not displayed in the console:

- Download and install python from software center (recommended)
- Alternatively, download and install python from [official website](#).

2. Install pre-commit:

```
pip install pre-commit
```

3. Open powershell (if already not opened)

```
cd <location to the tool base folder>
```

4. Run hooks:

```
pre-commit run --files $(git ls-files <path-to-tool-folder-with-files>/*)
```

5. Fix reported issues and re-run until all hooks pass.

Info

The pre-commit hooks in **SANDBOX** only provide suggestions about failing checks and potential improvements. They **do not prevent** commits from being made to the branch.

Freeze the development in SANDBOX

Once all the pre-commit checks are fixed and the tool is ready for migration:

- Freeze the tool in **SANDBOX** and no further development should be done here.
- Place a `README.md` stating that there will be no further development as the tool is being migrated and any further development will be taken as a feature request by DT.

Tool Placement in Codebase

- Determine the appropriate location for the tool within the codebase.
- The location of the tool should be finalized on alignment with **DT**

Note

Code location should be chosen in **engineering codebase**, aligned to either common functionality or tool-specific needs.

Git History

Final step before migration is to decide on whether the tool should be migrated with Git commit history.

- **Default Approach:** Is to migrate the tool **without** the commit history.
 - A strong and valid argument should be made if the Git history has to be retained (eg: critical past commits, traceability etc).
 - The final decision should be made in alignment with DT.
 - For more information on migration with Git-history please refer [how to migrate repo with history](#).
-

Summary Checklist

Task	Status
Recreate folder structure (DT standards)	☐
Document MATLAB + external dependencies	☐
Validate Zscaler compatibility	☐
Prepare ≥ 2 reference test cases	☐
Validate on Laptop + VDI + Eng VDI	☐
Create code coverage (if applicable)	☐
Run + fix pre-commit hooks	☐
Decide on Git history (justify if kept)	☐